

同花顺 3 年前端面经，期望薪资 25K

【一面（20min）】

- 1、vue 生命周期，dom 操作是在什么时候？（update）
- 2、mounted 和 created 区别
- 3、父子组件传值方法（3 种，props,this.parents.xx,vuex）
- 4、hash 和 history 模式区别
- 5、TCP 三次握手
- 6、OSI 七层模型
- 7、状态码（1xx,2xx,3xx,4xx,5xx）
- 8、队列和栈区别
- 9、JS 封装栈
- 10、跨域问题解决方法
- 11、cookie, localStorage, sessionStorage 区别
- 12、表单单数为红色，复数为蓝色，怎么实现？

【二面（20min）】

- 1、项目难点
- 2、OSI 协议层，五层与七层有啥区别？
- 3、冒泡排序，快排，快排稳定吗，时间复杂度多少，如果是 123456 快排复杂度多少？
- 4、有什么问我的？（问了技术栈，加班文化）

一面参考答案

1. Vue 生命周期，DOM 操作是在哪个时候？

Vue.js 的生命周期包括多个阶段，如创建（created）、挂载（mounted）、更新（updated）和销毁（destroyed）。DOM 操作通常在 **mounted** 和 **updated** 阶段进行，因为在这两个阶段，模板已经被渲染成真实的 DOM 元素。

2. mounted 和 created 区别

created：这个阶段发生在实例创建完成后，数据观测 (data observer) 和 event/watcher 事件配置之前。在这个阶段，组件的模板还没有被渲染成 DOM，因此无法进行 DOM 操作。

mounted：这个阶段发生在 el 被新创建的 vm.\$el 替换，并挂载到实例上去之后。在这个阶段，组件的模板已经被渲染成真实的 DOM 元素，可以进行 DOM 操作。

3. 父子组件传值方法

props：父组件通过 `props` 向子组件传递数据。

`**this.parent.xx**`：子组件可以通过 `'this.parent'` 访问父组件的实例，从而访问父组件的数据和方法。但这种方法不推荐，因为它破坏了组件的独立性。

Vuex：Vuex 是一个专为 Vue.js 应用程序开发的状态管理模式。它采用集中式存储管理应用的所有组件的状态，并以相应的规则保证状态以一种可预测的方式发生变化。

4. hash 和 history 模式区别

hash 模式：URL 中带有 `#` 符号。这种模式利用的是 URL 中的 hash 值（即 `#` 后面的部分）不会发送到服务器，只会在前端发生变化。因此，它不需要服务器进行任何配置。

history 模式：URL 中没有 `#` 符号，看起来更加简洁。这种模式利用了 HTML5 History API 来实现 URL 的跳转而不重新加载页面。但使用这种模式需要服务器进行配置，以便对所有的路由都返回同一个 HTML 文件。

5. TCP 三次握手

TCP 三次握手是建立可靠连接的过程：

1. 客户端发送一个 SYN 报文（同步序列编号）到服务器，并进入 SYN_SEND 状态。
2. 服务器收到 SYN 报文后，会回复一个 SYN+ACK 报文（同步序列编号 + 确认），并进入 SYN_RCVD 状态。

3. 客户端收到服务器的 SYN+ACK 报文后，会再次发送一个 ACK 报文（确认），此时客户端进入 ESTABLISHED 状态，服务器收到 ACK 报文后也进入 ESTABLISHED 状态，此时连接建立完成。

6. OSI 七层模型

OSI 七层模型从上到下分别是：

1. 应用层（Application Layer）
2. 表示层（Presentation Layer）
3. 会话层（Session Layer）
4. 传输层（Transport Layer）
5. 网络层（Network Layer）
6. 数据链路层（Data Link Layer）
7. 物理层（Physical Layer）

7. 状态码

1. 信息性状态码，表示接收到请求，正在处理。
2. 成功状态码，表示请求已成功被服务器接收、理解、并接受。
3. 重定向状态码，表示需要客户端采取进一步的操作才能完成请求。
4. 客户端错误状态码，表示请求包含语法错误或无法完成请求。
5. 服务器错误状态码，表示服务器在处理请求的过程中发生了错误。

8. 队列和栈区别

队列（Queue）：先进先出（FIFO, First In First Out）的数据结构。

栈（Stack）：先进后出（LIFO, Last In First Out）的数据结构。

9. JS 封装栈

在 JavaScript 中，可以使用数组来模拟栈的行为：

javascript

复制代码

```
1 class Stack {
2   constructor() {
3     this.items = [];
4   }
5
6   push(element) {
7     this.items.push(element);
8   }
9
10  pop() {
11    if (this.items.length === 0) {
12      return "Underflow";
13    }
14    return this.items.pop();
15  }
16
17  peek() {
18    return this.items[this.items.length - 1];
19  }
20
21  isEmpty() {
22    return this.items.length === 0;
23  }
24
25  size() {
26    return this.items.length;
27  }
28
29  printStack() {
30    console.log(this.items.toString());
31  }
32 }
```

10. 跨域问题解决方法

JSONP：只支持 GET 请求，通过在 URL 中添加回调函数名来接收数据。

CORS：通过设置服务器端的 CORS 头允许跨域请求。

代理服务器：通过设置一个代理服务器来转发请求，从而绕过跨域限制。

Nginx 反向代理：在 Nginx 中配置反向代理，将请求转发到目标服务器。

WebSocket：通过 WebSocket 协议进行实时通信，不受同源策略限制。

11. cookie, localStorage, sessionStorage 区别

cookie：存储在客户端浏览器，用于保持会话状态，有大小限制（通常为 4KB），且每次 HTTP 请求都会发送到服务器。

localStorage：存储在客户端浏览器中，没有过期时间，除非手动删除，否则会永久存储。

sessionStorage：存储在客户端浏览器中，页面会话结束时（通常是关闭页面时）会被清除。

12. 表单单数为红色，复数为蓝色，怎么实现？

可以使用 Vue.js 的数据绑定和条件渲染来实现这个功能。假设有一个表单输入字段用于输入数字，并根据输入的数字是单数还是复数来改变背景颜色：

```
html 复制代码
1 <div id="app">
2   <input type="number" v-model.number="number" @input="changeColor">
3   <div :style="{ backgroundColor: color }">
4     输入的数字是 {{ number }}
5   </div>
6 </div>
7
8 <script src="https://cdn.jsdelivr.net/npm/vue@2"></script>
9 <script>
10 new Vue({
11   el: '#app',
12   data: {
13     number: 0,
14     color: ''
15   },
16   methods: {
17     changeColor() {
18       this.color = this.number % 2 === 0 ? 'blue' : 'red';
19     }
20   }
21 });
22 </script>
```

在这个例子中，我们使用 Vue 的 `v-model` 指令来绑定输入框的值到 Vue 实例的 `number` 属性，并使用 `@input` 事件监听输入框的变化。当输入框的值变化时，调用 `changeColor` 方法，根据输入的数字是单数还是复数来改变 `color` 属性的值，从而改变背景颜色。